

Оглавление

- Типы кнопок
 - Тактовая
 - Большая
 - Микровыключатель
- Подключение к модулям
- Подключение к Arduino
 - Кнопка
 - Модуль
- Программирование
 - Нажатие и отпускание
 - Дребезг и помехи
 - Аппаратный дебаунс 1
 - Аппаратный дебаунс 2
 - Программный дебаунс 1
 - Программный дебаунс 2
 - Программный дебаунс 3
 - Удержание
 - Однократное удержание
 - Класс кнопки
 - События
- Библиотеки
 - Ввод
 - Кнопка
 - Сенсорная кнопка
 - Потенциометр
 - Инкрементальный энкодер
 - Джойстик
 - ИК пульт и приёмник
 - Матричная клавиатура
 - Bluetooth JDY-31
 - Радио модули 433 МГц
 - Считыватель RFID RC522
 - Индикация
 - Датчики
 - Актуаторы

Обучающие проекты 1

Уроки > Модули и компоненты > Ввод > Кнопка

Кнопка

Оглавление

- Типы кнопок
 - Тактовая
 - Большая
 - Микровыключатель
- Подключение к модулям
- Подключение к Arduino
 - Кнопка
 - Модуль
- Программирование
 - Нажатие и отпускание
 - Дребезг и помехи
 - Аппаратный дебаунс 1
 - Аппаратный дебаунс 2
 - Программный дебаунс 1
 - Программный дебаунс 2
 - Программный дебаунс 3
 - Удержание
 - Однократное удержание
 - Класс кнопки
 - События
- Библиотеки

Кнопка – самый базовый элемент ввода данных в микроконтроллер, используется для управления электронным устройством. Функционально кнопка замыкает контакт при нажатии, что может быть использовано для передачи через неё электрического сигнала на пин МК - он читает значение пина и таким образом понимает, нажата кнопка или нет.

	В наборе GyverKIT	Aliexpress
	START IOT	Купить
	START IOT	Купить
	START IOT	Купить

> Типы кнопок

Фиксация:

- Без фиксации (*momentary*) - после отпускания отключается обратно
- С фиксацией (*latching*) - после отпускания остаётся нажатой

Поведение:

- Нормально разомкнутая (*Normal Open, NO*) - при нажатии замыкает контакты



- **Нормально замкнутая** (*Normal Closed, NC*) - при нажатии размыкает контакты

Оглавление

Типы кнопок

Тактовая

Большая

Микровыключатель

Подключение к модулям

Подключение к Arduino

Кнопка

Модуль

Программирование

Нажатие и отпускание

Дребезг и помехи

Аппаратный дебаунс 1

Аппаратный дебаунс 2

Программный дебаунс 1

Программный дебаунс 2

Программный дебаунс 3

Удержание

Однократное удержание

Класс кнопки

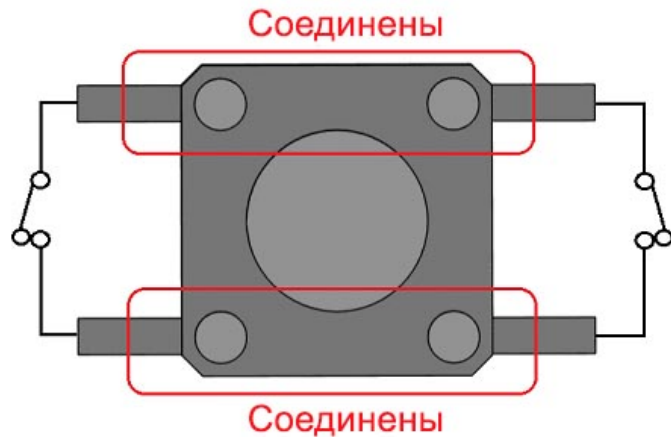
[События](#)

Библиотеки

×

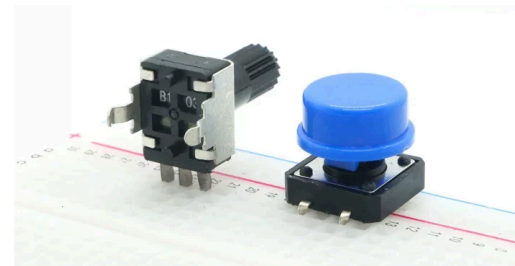
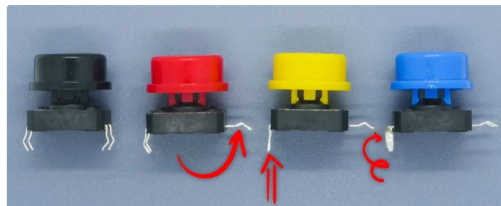
Тактовая

Самая простая "стандартная" кнопка, используется повсеместно. Без фиксации, нормально разомкнутая. Бывает разных размеров, стандарт 6x6 мм:



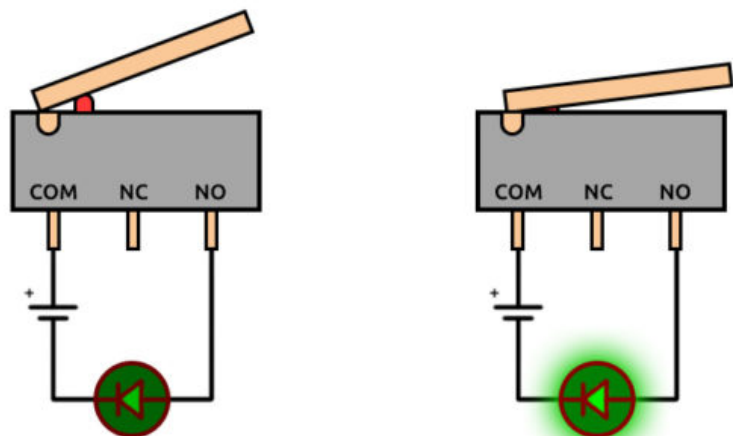
Большая

К большим тактовым кнопкам (12x12 мм) есть специальные колпачки. Большие кнопки не очень приспособлены для подключения на макетной плате: у них очень широкие изогнутые ножки, которые туго входят в отверстия. Можно выпрямить их, а затем скрутить на 90 градусов при помощи пинцета. Так они не будут излишне растягивать контакты платы. Вторую пару ножек можно отогнуть или откусить кусачками:



Микровыключатель

"Микрик" обычно имеет три контакта, общий **COM**, нормально открытый **NO** и нормально закрытый **NC**. При отпущенной кнопке замкнута цепь **COM-NC**, при нажатой замыкается **COM-NO**. Часто встречаются там, где кнопка нажимается частью механизма (концевик станка, принтера, дверца микроволновки)

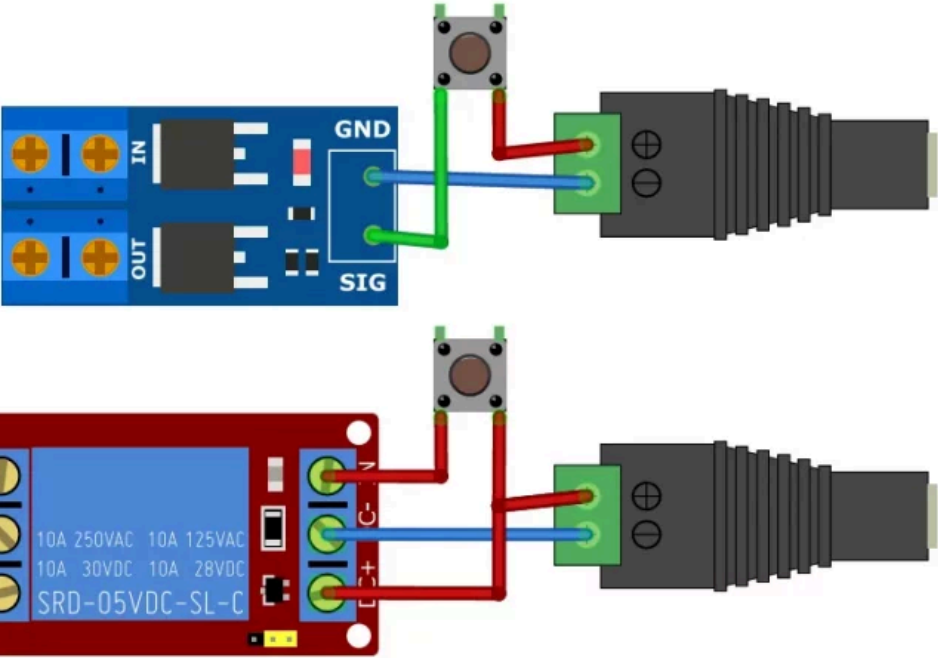


Подключение к модулям

Кнопка может быть использована для управления модулями с логическим входом: реле, транзистор и другим подобным. Рассмотрим подключение при питании от сетевого адаптера:

Оглавление

- Типы кнопок
 - Тактовая
 - Большая
 - Микровыключатель
- Подключение к модулям
- Подключение к Arduino
 - Кнопка
 - Модуль
- Программирование
 - Нажатие и отпускание
 - Дребезг и помехи
 - Аппаратный дебаунс 1
 - Аппаратный дебаунс 2
 - Программный дебаунс 1
 - Программный дебаунс 2
 - Программный дебаунс 3
 - Удержание
 - Однократное удержание
- Класс кнопки
 - События
- Библиотеки

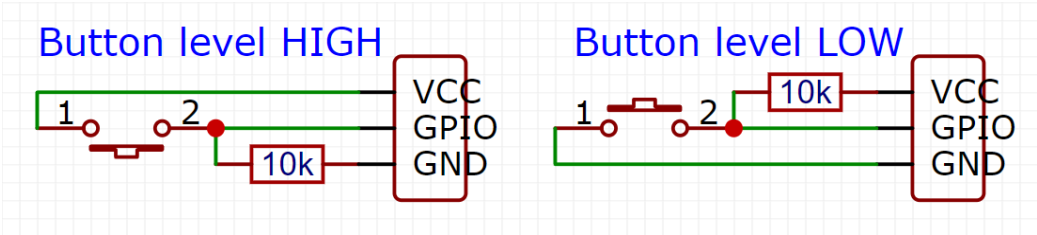


Подключение к Arduino

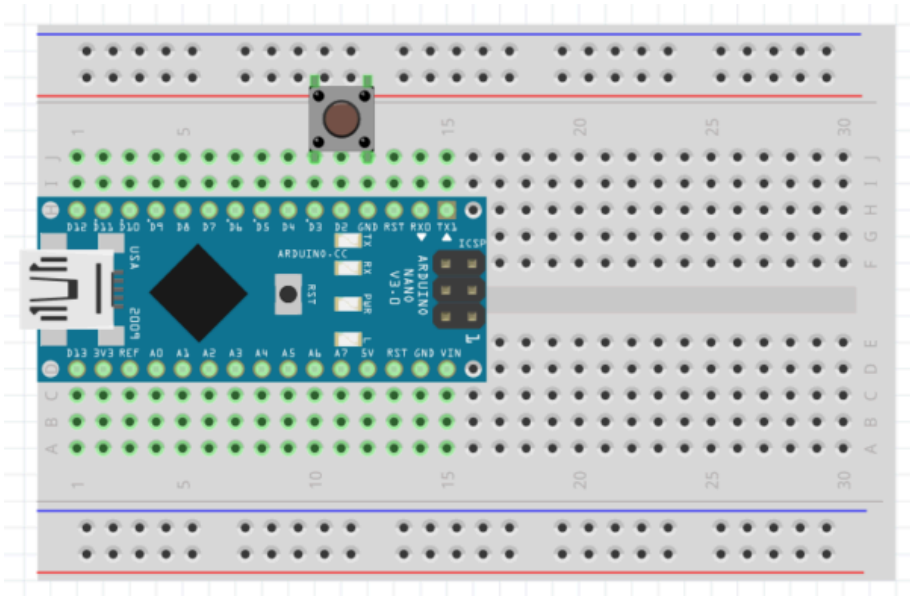
Кнопка

Рекомендуется изучить урок по работе с цифровыми входами

Кнопка замыкает два контакта, поэтому пин МК должен быть подтянут к противоположному полюсу питания резистором ~10 кОм:



В Arduino мы можем подключить кнопку к GND и использовать внутреннюю подтяжку пина INPUT_PULLUP :



Оглавление

Типы кнопок

 Тактовая

 Большая

 Микровыключатель

Подключение к модулям

Подключение к Arduino

 Кнопка

 Модуль

Программирование

 Нажатие и отпускание

 Дребезг и помехи

 Аппаратный дебаунс 1

 Аппаратный дебаунс 2

 Программный дебаунс 1

 Программный дебаунс 2

 Программный дебаунс 3

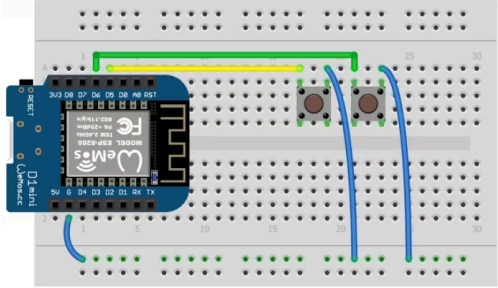
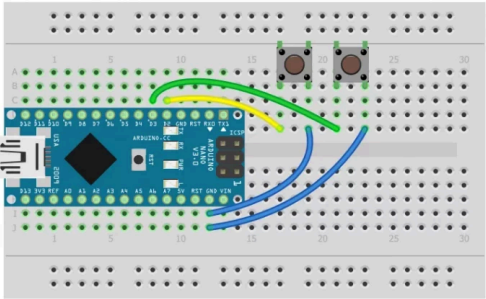
 Удержание

 Однократное удержание

 Класс кнопки

 События

Библиотеки



```
#define BUTTON_PIN 3
```

```
void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
    bool state = !digitalRead(BUTTON_PIN); // инвертируем
    Serial.println(state);                // 1 - нажата, 0 - отпущена
    delay(100);
}
```



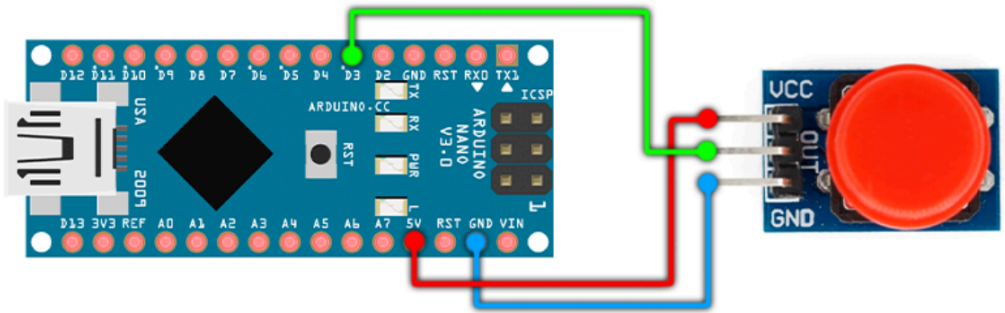
Несмотря на инвертированную логику нажатия, вариант с подтяжкой к плюсу питания используется на практике очень часто по целому ряду причин:

- В большинстве МК есть встроенная подтяжка к питанию, чего не скажешь о подтяжке к GND
- GND является общей точкой схемы и почти всегда есть рядом, поэтому к кнопке нужно "тащить" только вывод на GPIO
- Разводить печатную плату по этой же причине гораздо проще и удобнее
- Замыкание сигнального провода на GND безопасно (в сценарии где от кнопки отвалился провод и упал на плату или корпус)
- Это естественная open-drain логика, низкий активный уровень исторически очень распространён во многих микросхемах и шинах

➤ Модуль

Модуль подключается к питанию и цифровому пину. На плате пин кнопки **подтянут к GND** резистором:

- При нажатии кнопка выдаёт сигнал **HIGH**
- Режим работы пина должен быть **INPUT** , а не с подтяжкой к питанию, как у обычной кнопки!



```
#define BUTTON_PIN 3
```

```
void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT);
}

void loop() {
    bool state = digitalRead(BUTTON_PIN);
    // 1 - нажата, 0 - отпущена
}
```



Оглавление

Типы кнопок

Тактовая

Большая

Микровыключатель

Подключение к модулям

Подключение к Arduino

Кнопка

Модуль

Программирование

Нажатие и отпускание

Дребезг и помехи

Аппаратный дебаунс 1

Аппаратный дебаунс 2

Программный дебаунс 1

Программный дебаунс 2

Программный дебаунс 3

Удержание

Однократное удержание

Класс кнопки

События

Библиотеки

```
Serial.println(state);
delay(100);
}
```

Программирование

Нажатие и отпускание

Чтобы разделить нажатие и отпускание на два отдельных события, используем флаг (машина состояний с двумя состояниями):

```
#define BUTTON_PIN 3

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  static bool pState = false;
  bool state = !digitalRead(BUTTON_PIN);
  if (pState != state) { // состояние изменилось
    pState = state;      // запомнить новое
    if (state) Serial.println("Кнопка нажата");
    else Serial.println("Кнопка отпущена");
  }
}
```

Дребезг и помехи

Кнопка не идеальна и контакт замыкается не сразу - какое-то время он механически "дребезжит" (*bounce*) - внутри кнопки находится металлическая пластинка, которая колеблется при нажатии и отпускании. Прогоняя данный алгоритм, система опрашивает кнопку и условия приблизительно за 6 мкс, то есть кнопка опрашивается около 166'666 раз в секунду! Этого достаточно, чтобы получить несколько тысяч "нажатий" вместо одного:



Нажатие кнопки. При отпускании получится аналогичная картина

Кнопка нажата
Кнопка отпущена
Кнопка нажата
Кнопка отпущена
Кнопка нажата

Для однозначного определения состояния кнопки дребезг нужно "погасить" - **debounce**.

💡 Организация антидребезга также поможет избежать помех, которые могут приводить к ложным нажатиям кнопки. Например, при длинных проводах до кнопки, при использовании внутренней подтяжки пина и наличии рядом индуктивной нагрузки (мотор, реле, электромагнит, соленоид, провод с током), особенно если кнопка подключена на аппаратное прерывание МК

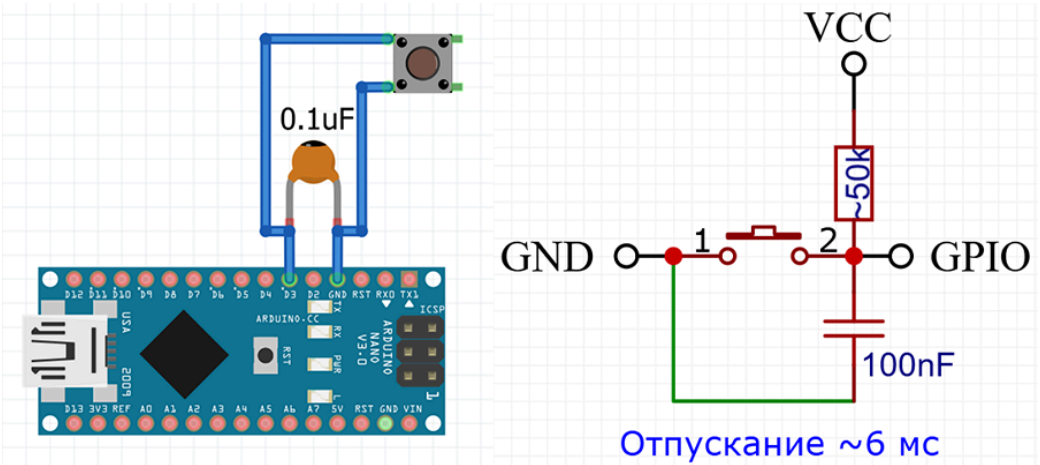
Дребезг и помехи можно гасить аппаратно - при помощи RC фильтра, образованного резистором и конденсатором, а можно программно - опрашивая кнопку по времени.

×

➤ Аппаратный дебаунс 1

	В наборе GyverKIT	Aliexpress
	START IOT	Купить
	START IOT	Купить
	START	Купить

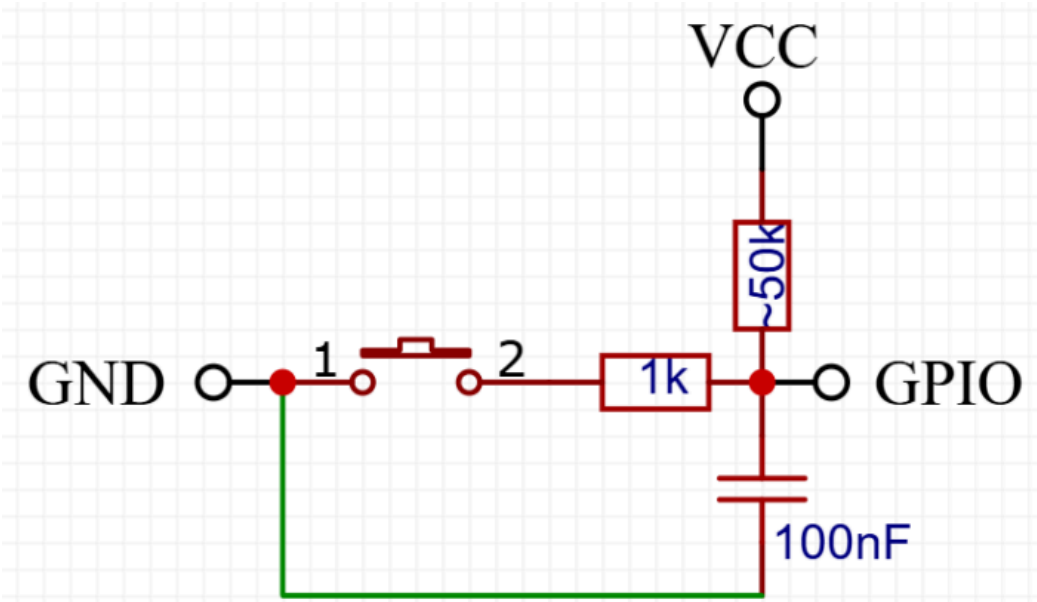
Самая простая схема - асимметричная. При использовании внутренней подтяжки достаточно добавить в схему конденсатор, который будет медленно разряжаться через резистор подтяжки:



В данной схеме нажатие кнопки даёт мгновенный сигнал на GPIO, а отпускание происходит плавно. Если кнопка дребезжит - медленный рост сигнала при отпускании не даёт помехам подняться выше логического уровня и дребезг гасится. Но эта схема не защищает от помех, т.к. "нажатие" происходит без задержки, а сопротивление подтяжки мало - выброс от мотора или другой катушки неподалёку может запросто "прижать" такую кнопку. Для увеличения времени дебаунса можно увеличить ёмкость конденсатора, но тогда лучше добавить ещё один защитный резистор примерно на 1 кОм - конденсатор ведь разряжается прямо в кнопку:

Оглавление

- Типы кнопок
 - Тактовая
 - Большая
 - Микровыключатель
- Подключение к модулям
 - Кнопка
 - Модуль
- Программирование
 - Нажатие и отпускание
 - Дребезг и помехи
 - Аппаратный дебаунс 1
 - Аппаратный дебаунс 2
 - Программный дебаунс 1
 - Программный дебаунс 2
 - Программный дебаунс 3
 - Удержание
 - Однократное удержание
- Класс кнопки
 - События
- Библиотеки

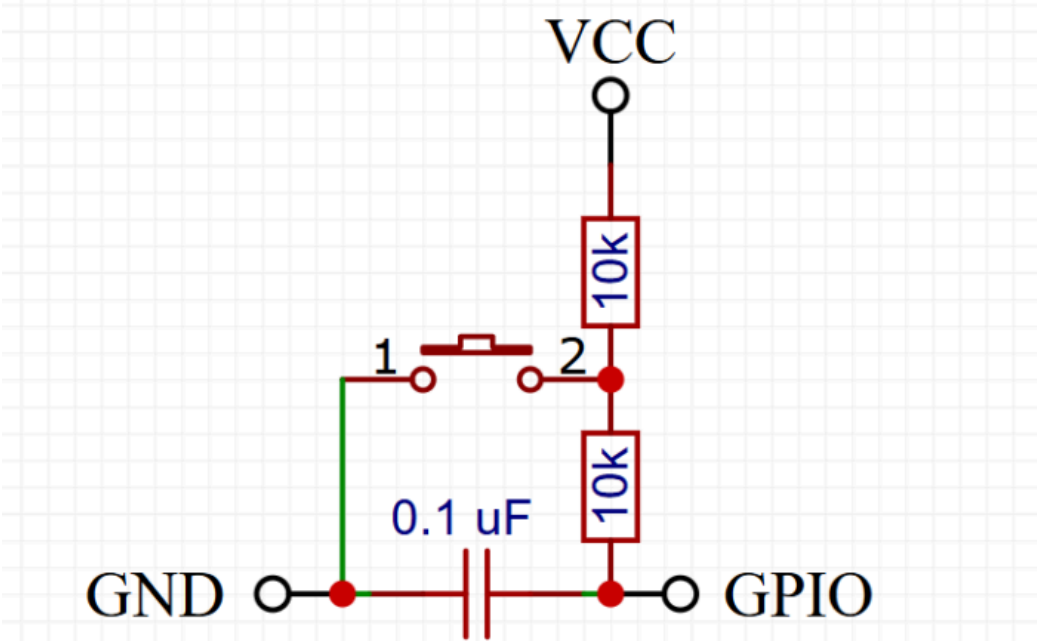


Нажатие ~0.1 мс, отпускание ~6 мс

Здесь уже образуется полноценная RC цепь и нажатие тоже будет сглаживаться. Но если уж мы добавили резистор, то лучше добавить ещё один и сделать более симметричную схему.

➤ Аппаратный дебаунс 2

Убираем внутреннюю подтяжку и заменяем её внешней, со стабильным и не таким высоким сопротивлением, как у внутренней - это будет надёжнее:



Нажатие ~1.2 мс, отпускание ~2.4 мс

Данная схема сглаживает и нажатие, и отпускание, в реальных проектах лучше применять её (или ещё более надёжные схемы). С указанными номиналами схема сгладит мелкий дребезг и помехи, но лучше добавить дополнительный программный антидребезг на 20-40 мс - для кнопки, нажимаемой человеком. Также можно взять 1 мкФ конденсатор вместо 0.1 - тогда время гашения увеличится в 10 раз (до 12 и 24 мс) и будет полноценным аппаратным гашением дребезга, что важно в случаях, когда кнопка подключена на прерывания и/или пробуждает устройство из режима сна.

➤ Программный дебаунс 1

Гашение дребезга можно сделать и программно. Самый простой вариант - опрашивать кнопку не постоянно в цикле, а периодически, например каждые 50 мс (20 раз в секунду) - вполне достаточно для того, чтобы "пропустить" дребезг даже от самой плохой кнопки и снизить вероятность поймать внешние "помехи" почти до нуля: ▲

Оглавление

Типы кнопок

Тактовая

Большая

Микровыключатель

Подключение к модулям

Подключение к Arduino

Кнопка

Модуль

Программирование

Нажатие и отпускание

Дребезг и помехи

Аппаратный дебаунс 1

Аппаратный дебаунс 2

Программный дебаунс 1

Программный дебаунс 2

Программный дебаунс 3

Удержание

Однократное удержание

Класс кнопки

События

Библиотеки

×

```
void loop() {
    static bool pState = false;
    bool state = !digitalRead(BUTTON_PIN);

    if (pState != state) { // состояние изменилось
        pState = state;    // запомнить новое
        if (state) Serial.println("Кнопка нажата");
        else Serial.println("Кнопка отпущена");
    }

    delay(50); // ждём
}
```

Этот вариант очевидно плох наличием задержки, которая будет тормозить всю программу, поэтому лучше опрашивать кнопку по таймеру, например - по таймеру на миллис :

```
#define BUTTON_PIN 3
#define BTN_DEB 50 // тайм-аут

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
    static uint32_t tmr;
    // таймер
    if (millis() - tmr >= BTN_DEB) {
        tmr = millis();

        // далее тот же код, что выше
        static bool pState = false;
        bool state = !digitalRead(BUTTON_PIN);
        if (pState != state) { // состояние изменилось
            pState = state;    // запомнить новое
            if (state) Serial.println("Кнопка нажата");
        }
    }
}
```

💡 Самый простой вариант - реагирует на изменение состояния с задержкой и с небольшой вероятностью может пропустить помеху

Данный вариант хорош тем, что "из коробки" работает для энергоэффективных систем - реальное устройство с батарейным питанием может уходить "в сон" между опросами кнопки. Это самый простой вариант, без пробуждения по нажатию и так далее.

➤ Программный дебаунс 2

Более интересный вариант - ловим первое изменение сигнала пина и игнорируем остальные, пока идёт таймаут:

```
#define BUTTON_PIN 3
#define BTN_DEB 50 // тайм-аут смены состояния, мс

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
    static bool pState = false;
    static uint32_t tmr;

    bool state = !digitalRead(BUTTON_PIN);
    // состояние изменилось и вышел таймер
    if (pState != state && millis() - tmr >= BTN_DEB) {
        tmr = millis(); // сбросить таймер
        pState = state;  // запомнить состояние
        if (state) Serial.println("Кнопка нажата");
        else Serial.println("Кнопка отпущена");
    }
}
```

Оглавление

Типы кнопок

Тактовая

Большая

Микровыключатель

Подключение к модулям

Подключение к Arduino

Кнопка

Модуль

Программирование

Нажатие и отпускание

Дребезг и помехи

Аппаратный дебаунс 1

Аппаратный дебаунс 2

Программный дебаунс 1

Программный дебаунс 2

Программный дебаунс 3

Удержание

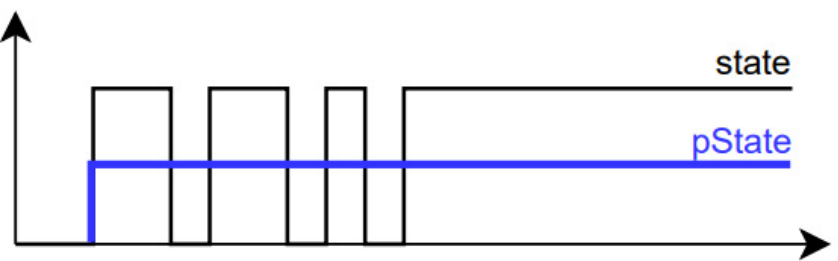
Однократное удержание

Класс кнопки

События

Библиотеки

×



💡 Этот вариант **мгновенно реагирует** на нажатие и отпускание, но может пропустить "помеху" - короткий скачок напряжения. По сути - программный аналог "аппаратного дебаунса 1"

➤ Программный дебаунс 3

Ещё более интересный вариант - ФНЧ (фильтр низких частот) - виртуальное состояние **pState** не изменяется, пока не выйдет таймаут при стабильном реальном состоянии. Переменная таймера также используется в качестве флага первого изменения (значение **0**). Это уже аналог "аппаратного дебаунса 2" и работает в обе стороны:

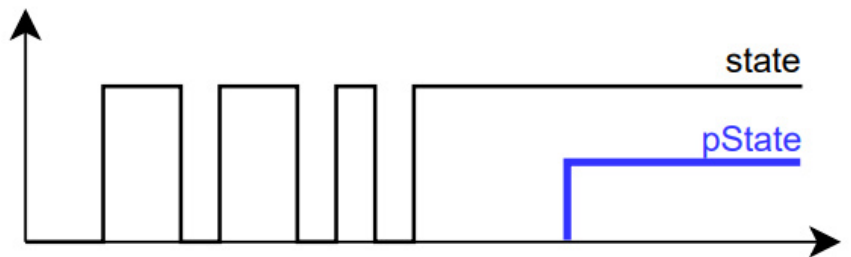
```
#define BUTTON_PIN 3
#define BTN_DEB 50 // тайм-аут смены состояния, мс

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
    static bool pState = false;
    static uint32_t tmr;
    bool state = !digitalRead(BUTTON_PIN);

    if (pState != state) {
        if (!tmr) tmr = millis(); // первое изменение
        else if (millis() - tmr >= BTN_DEB) { // вышел тайм-аут
            pState = state; // запомнить состояние
        }

        if (state) Serial.println("Кнопка нажата");
        else Serial.println("Кнопка отпущена");
    }
}
```



💡 Этот вариант получает изменение не сразу, но позволяет **игнорировать помехи** в проводе

Во всех случаях дребезг контактов должен пропасть - кнопка будет выдавать два чётких события при нажатии и отпускании.

➤ Удержание

Добавив ещё одну конструкцию таймера можно получить событие удержания и импульсного удержания - кнопка удерживается и "сигналил" событиями с заданным периодом:

```
#define BUTTON_PIN 3
#define BTN_DEB 50 // тай-маут смены состояния, мс
#define BTN_HOLD 500 // тай-маут удержания, мс

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}
```

⬆

Оглавление

- Типы кнопок
 - Тактовая
 - Большая
 - Микровыключатель
- Подключение к модулям
- Подключение к Arduino
 - Кнопка
 - Модуль
- Программирование
 - Нажатие и отпускание
 - Дребезг и помехи
 - Аппаратный дебаунс 1
 - Аппаратный дебаунс 2
 - Программный дебаунс 1
 - Программный дебаунс 2
 - Программный дебаунс 3
 - Удержание
 - Однократное удержание
 - Класс кнопки
 - События
- Библиотеки



```
void loop() {
    static bool pState = false;
    static uint32_t tmr;

    bool state = !digitalRead(BUTTON_PIN);
    if (pState != state && millis() - tmr >= BTN_DEB) { // дебаунс 2 типа
        tmr = millis();
        pState = state;
        if (state) Serial.println("Кнопка нажата");
        else Serial.println("Кнопка отпущена");
    }
```

При нажатии и удержании кнопки данный код сначала выведет "Кнопка нажата" затем начнёт выводить "Кнопка удержана" с периодом 500 мс - примерно так реализовано изменение "настройки" при клике и удержании в большинстве электронных приборов.

Однократное удержание

Если нужен **однократный** сигнал на удержание - можно ввести флаг:

```
#define BUTTON_PIN 3
#define BTN_DEB 50 // тай-маут смены состояния, мс
#define BTN_HOLD 700 // тай-маут удержания, мс

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
    static bool pState = false;
    static bool hold = false; // флаг удержания
    static uint32_t tmr;

    bool state = !digitalRead(BUTTON_PIN);
    if (pState != state && millis() - tmr >= BTN_DEB) {
        tmr = millis();
        pState = state;
        hold = false; // сброс флага удержания
        if (state) Serial.println("Кнопка нажата");
    }
```

Класс кнопки

Примеры выше рассчитаны на обработку **одной** кнопки, чтобы показать саму суть алгоритмов. Если нужно обрабатывать больше - код нужно обернуть в более универсальный модуль, например в класс. Примеры в других уроках на сайте:

- Разделение на модули - самый простой вариант с обработкой нажатия и гашениемдребезга
- Конечный автомат - большой конечный автомат с обработкой клика, удержания, импульсного режима и отпускания отдельно из всех режимов

Напишем класс на основе алгоритмов из текущего урока. События клика и удержания будем запоминать в отдельных флагах и проверять через методы, а опрос кнопки сделаем в тикере, который нужно вызывать в основном цикле:

main.cppButton.h

```
#include "Button.h"

// кнопки подключены к пинам 3 и 4 и GND
Button btn1(3);
Button btn2(4);

void setup() {
    Serial.begin(115200);
}

void loop() {
    btn1.tick();
    if (btn1.click()) Serial.println("click 1");
    if (btn1.hold()) Serial.println("hold 1");
}
```



Оглавление

Типы кнопок

Тактовая

Большая

Микровыключатель

Подключение к модулям

Подключение к Arduino

Кнопка

Модуль

Программирование

Нажатие и отпускание

Дребезг и помехи

Аппаратный дебаунс 1

Аппаратный дебаунс 2

Программный дебаунс 1

Программный дебаунс 2

Программный дебаунс 3

Удержание

Однократное удержание

Класс кнопки

События

Библиотеки



```
btn2.tick();
if (btn2.click()) Serial.println("click 2");
if (btn2.hold()) Serial.println("hold 2");
}
```

События

Можно переписать класс в событийно-ориентированном стиле, добавив подключения обработчиков. В этом случае не придётся делать манипуляции с флагами, но придётся хранить обработчики:

main.cpp Button.h

```
#include "Button.h"

// кнопки подключены к пинам 3 и 4
Button btn1(3);
Button btn2(4);

void click1() {
    Serial.println("click 1");
}
void hold1() {
    Serial.println("hold 1");
}

void setup() {
    Serial.begin(115200);

    btn1.onClick(click1);
    btn1.onHold(hold1);

    btn2.onClick(click2);
    btn2.onHold(hold2);
}
```

Библиотеки

Для удобной работы с кнопками можно использовать библиотеки, например `EncButton`, её можно установить/обновить из встроенного менеджера библиотек Arduino по названию **EncButton**. Краткая документация находится по ссылке выше, базовые примеры есть в самой библиотеке. У `EncButton` есть аналог с более читаемым исходником - библиотека `uButton`.

Для работы библиотеки нужно вызывать метод `tick()` в `loop()` и опрашивать нужные события. Их там много - смотрите документацию.

- Демо
- Кнопка управляет светодиодом
- Две кнопки, разные действия
- Две кнопки меняют значение переменной
- Кнопка управляет яркостью светодиода

Полезные страницы

- Набор GyverKIT – наш большой стартовый набор Arduino, продаётся в России
- Каталог ссылок на дешёвые Ардуины, датчики, модули и прочие железки с AliExpress
- Обратная связь – сообщить об ошибке в уроке или предложить дополнение по тексту (alex@alexgyver.ru)
- Поддержать автора за работу над уроками



Оглавление

Типы кнопок

Тактовая

Большая

Микровыключатель

Подключение к модулям

Подключение к Arduino

Кнопка

Модуль

Программирование

Нажатие и отпускание

Дребезг и помехи

Аппаратный дебаунс 1

Аппаратный дебаунс 2

Программный дебаунс 1

Программный дебаунс 2

Программный дебаунс 3

Удержание

Однократное удержание

Класс кнопки

События

Библиотеки



Обновлено 15.07.2026

← Считыватель RFID RC522

Сенсорная кнопка →

✉ Подписаться ▼



Присоединиться к обсуждению

B
I
U
☞
☰
☷
”
</>
☞
{ }
[+]

32 КОММЕНТАРИЕВ

⚡ 🔥 Старые ▼

**антон** ⌚ 1 год назад

а как сделать так что бы светодиод мигал по нажатию кнопки один раз нажал и он сам мигает

^ 0 ▼ ➞ Ответить

Просмотр ответов (1) ▼

**Артем** ⌚ 1 год назад

ОШИБКА!!! В примере — демо библиотеки не запущен монитор порта

^ 0 ▼ ➞ Ответить

Просмотр ответов (1) ▼

**Егор** ⌚ 1 год назад

Скажите а INPUT_PULLUP на Attiny88 например работает ?

^ 0 ▼ ➞ Ответить

Просмотр ответов (1) ▼

**NAZAR** ⌚ 1 год назад

а можно как то реализовать чтоб одна кнопка переключала режимы по очереди?

^ 0 ▼ ➞ Ответить

Просмотр ответов (3) ▼

**Андрей** ⌚ 11 месяцев назад

Добрый день. Вопрос по примеру «Однократное удержание». Правильно ли я понял, что в первоначальный момент условие «if (pState != state && millis() — tmr >= BTN_DEB)» будет истинно при смене состояния кнопки независимо от millis() — tmr >= BTN_DEB? Т.е. программа не будет ждать BTN_DEB при смене состояния, а сразу выдаст сообщение «Кнопка нажата»?

^ 0 ▼ ➞ Ответить

Просмотр ответов (6) ▼

**Игорь** ⌚ 11 месяцев назад

А как сделать так что бы кнопку просто замыкала цепь, я не понял Я имею в виду «ты нажал на кнопку и светодиод загорелся»

^ 0 ▼ ➞ Ответить

Просмотр ответов (1) ▼

**Иван** ⌚ 10 месяцев назад

А как реализовать механизм кнопки с фиксатором, чтобы на кнопку нажали один раз и при определённом условии (через 10 секунд) она самостоятельно отжимается?

^ 0 ▼ ➞ Ответить

Просмотр ответов (2) ▼

**dmitriy** ⌚ 7 месяцев назад

использую pinMode(2, INPUT_PULLUP). Не всегда срабатывает кнопка, решил плохое качество кнопки, начал пробовать по касанию проводов и понял дело не в кнопке, чтобы зафиксировать «наличие контакта» иногда необходимо

Оглавление

- Типы кнопок
 - Тактовая
 - Большая
 - Микровыключатель
- Подключение к модулям
- Подключение к Arduino
 - Кнопка
 - Модуль
- Программирование
 - Нажатие и отпускание
 - Дребезг и помехи
 - Аппаратный дебаунс 1
 - Аппаратный дебаунс 2
 - Программный дебаунс 1
 - Программный дебаунс 2
 - Программный дебаунс 3
 - Удержание
 - Однократное удержание
 - Класс кнопки
- События
- Библиотеки



больше времени. Т.е. иногда молниеносно происходит переключение, а иногда необходимо «подержать». Этому есть объяснение?

```
void setup() { Serial.begin(9600); pinMode(2, INPUT_PULLUP); pinMode(10, OUTPUT); }

bool button; //переменная для памяти значения кнопки
void loop() {
  if (digitalRead(2)==LOW){ //фиксируем нажатие кнопки
    button = !button; //меняем значение кнопки на противоположное
    if (button==1) { digitalWrite(10, 150); } //подаем на диод примерно 3вольта
    else { digitalWrite(10, LOW); } //или выключаем диод
  }
  Serial.print(button); //вывод в терминал, это проверял исправность по линии
  включения диода
  delay(500); //защита отдребезга кнопки
}
```

🔗 Последний раз редактировалось 7 месяцев назад dmitriy em

0 Ответить Просмотр ответов (4)



Vlad 5 месяцев назад
Алекс — чертов гений! Завидую черно-бело-малиновой завистью...

0 Ответить



Евгений 4 месяцев назад
В примере «Две кнопки, разные действия» опечатка — held (hold).

0 Ответить Просмотр ответов (1)



Владимир 1 месяц назад

```
#define BUTTON_PIN 3

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  static bool pState = false;
  bool state = !digitalRead(BUTTON_PIN);
  if (pState != state) { // состояние изменилось
    pState = state; // запомнить новое
    if (state) Serial.println("Кнопка нажата");
    else Serial.println("Кнопка отпущена");
  }
}
```

1) Переменная pState в данной строке «static bool pState = false;» получается не перезаписывается (принимает значение false) каждый раз при прохождении основного цикла? Я так понял ей помогает слово static.

0 Ответить Просмотр ответов (1)

